

MERN STACK

DAY – 18

15 July 2026

React

TASK-1

```
import { useState } from "react";

export const Us = () => {
  const [name, setname] = useState("");
  const [na, setna] = useState([]);
  const [age, setage] = useState("");

  const ad = () => {
    const data = { name, age };
    setna([...na, data]);
  };

  return (
    <div className="container mt-4">
      <h3 className="text-center">User Details</h3>
      <input className="form-control mb-2" type="text" placeholder="Enter Name" onChange={(e) => setname(e.target.value)} />
      <input className="form-control mb-2" type="text" placeholder="Enter Age" onChange={(e) => setage(e.target.value)} />
      <button className="btn btn-primary mb-3" onClick={ad} >
        ADD
      </button>
      <table className="table table-bordered">
        <thead>
          <tr>
            <th>Name</th>
            <th>Age</th>
          </tr>
        </thead>
        <tbody>
          {
            na.map((a, index) => (
              <tr key={index}>
                <td>{a.name}</td>
                <td>{a.age}</td>
              </tr>
            ))
          }
        </tbody>
      </table>
    </div>
  );
};
```

User Details

ADD

Name	Age
Anjali	21
Anhad	21

TASK – 2 TODO LIST

A Todo List is a task management application used to add and display tasks that need to be completed. In React, a Todo List is created using components, state management, and event handling.

Concepts Used:

1. React Component

- A reusable piece of UI that contains logic and structure.

By : Aditi Tangri

URN : 2302460

CRN : 2315004

- The Todo List is created as a functional component.

2. useState Hook

- Used to create and update state variables.
- It allows React components to store changing data.

3. Event Handling

- Events like onChange and onClick are used to handle user actions.
- onChange captures input values.
- onClick adds the task to the list.

4. Array State Management

- Todo items are stored in an array.
- The spread operator (...) is used to add new tasks without removing existing tasks.

5. map() Method

- Used to iterate through the todo array.
- Each task is displayed dynamically in the table.

```
import { useState } from "react";

export const Todo = () => {
  const [task, setTask] = useState("");
  const [da, setDa] = useState("");
  const [todo, setTodo] = useState([]);

  const ab = () => {
    const data = { task, da };
    setTodo([...todo, data]);
  };

  return (
    <div className="container mt-4">
      <h3 className="text-center mb-3">Todo List</h3>
      <input className="form-control mb-2" type="text" placeholder="Enter Task" onChange={(e) => setTask(e.target.value)} />
      <input className="form-control mb-2" type="date" onChange={(e) => setDa(e.target.value)} />
      <button className="btn btn-success mb-3" onClick={ab} > ADD TASK</button>
      <table className="table table-bordered table-striped">
        <thead className="table-dark">
          <tr>
            <th>TASK</th>
            <th>DATE</th>
          </tr>
        </thead>
        <tbody>
          {
            todo.map((a, index) => (
              <tr key={index}>
                <td>{a.task}</td>
                <td>{a.da}</td>
              </tr>
            ))
          }
        </tbody>
      </table>
    </div>
  );
};
```

Todo List

CLEAN ROOMS

15-07-2026

TASK	DATE
WAKE UP EARLY	2026-07-16
READ A BOOK	2026-07-17
CHECK MAILS	2026-07-17
CLEAN ROOMS	2026-07-15

useEffect Hook

useEffect is a React Hook that allows you to perform side effects in functional components. A side effect is any operation that happens outside the normal component rendering process, such as:

- Showing an alert
- Fetching data from an API
- Changing the document title
- Setting timers (setTimeout, setInterval)
- Adding/removing event listeners
- Updating local storage

useEffect runs after the component renders.

Syntax:

```
useEffect(() => {  
  // side effect code  
}, [dependency]);
```

Dependency Array Behavior:

Code	When it runs
useEffect(() => {})	Runs after every render
useEffect(() => {}, [])	Runs only once when component loads
useEffect(() => {}, [state])	Runs when that state changes

```
import { useEffect, useState } from "react";

export const Ue=()=>{
  const [us,setus]=useState("")
  const [u,setusa]=useState("");
  useEffect(()=>{
    alert("this is first alert")
  })
  useEffect(()=>{
    alert("this is second alert")
  },[])
  useEffect(()=>{
    alert("this is third alert")
  },[us])

  return (
    <>
    <input type="text" onChange={(e)=>setus(e.target.value)}/>
    <input type="text" onChange={(e)=>setusa(e.target.value)}/>
    </>
  );
}
```

When the page loads:

localhost:3000 says
this is first alert

localhost:3000 says
this is second alert

localhost:3000 says
this is third alert

When typing in first input:

localhost:3000 says
this is first alert

localhost:3000 says
this is third alert

When typing in second input:

localhost:3000 says
this is first alert

TASK – 3 COUNTER

```
import { useState } from "react";

export const Us = () => {
  const [count, setcount] = useState(0);

  const ad = () => {
    let v = count + 1;
    setcount(v);
  };
  const mi = () => {
    let v = count - 1;
    setcount(v);
  };
  return (
    <div className="container mt-5">
      <div className="card shadow p-4 mx-auto text-center" style={{maxWidth:"400px"}}>
        <h2 className="text-primary mb-4"> COUNTER </h2>
        <div className="border rounded bg-light p-3 mb-4">
          <h1 className="display-3 fw-bold text-dark">
            {count}
          </h1>
        </div>
        <div className="d-flex justify-content-center gap-3">
          <button
            className="btn btn-success btn-lg"
            onClick={ad}
          >
            + INCREMENT
          </button>
          <button
            className="btn btn-danger btn-lg"
            onClick={mi}
          >
            - DECREMENT
          </button>
        </div>
      </div>
    </div>
  );
};
```

